

POSIX Threads (pthread) Race Condition and Mutex Demonstration

Multithreaded C program showing an incorrect counter value caused by a race condition and the corrected version using a mutex.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

long long counter = 0;
pthread_mutex_t mutex;

/* -----
   Version 1: No Synchronization
   ----- */
void *increment_without_mutex(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        counter++; // Race condition
    }
    return NULL;
}

/* -----
   Version 2: Using Mutex
   ----- */
void *increment_with_mutex(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        pthread_mutex_lock(&mutex);

        counter++; // Critical section

        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    /* ----- Race Condition Demo ----- */
    counter = 0;

    printf("Running WITHOUT mutex...\n");

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL,
                      increment_without_mutex, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
}
```

```

    }

    printf("Expected Counter = %d\\n",
          NUM_THREADS * INCREMENTS);
    printf("Actual Counter   = %lld\\n\\n", counter);

    /* ----- Mutex Demo ----- */
    counter = 0;

    pthread_mutex_init(&mutex, NULL);

    printf("Running WITH mutex...\\n");

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL,
                      increment_with_mutex, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected Counter = %d\\n",
          NUM_THREADS * INCREMENTS);
    printf("Actual Counter   = %lld\\n", counter);

    pthread_mutex_destroy(&mutex);

    return 0;
}

/*
Compilation:
gcc -pthread race_condition.c -o race_condition

```

Example Output:

Running WITHOUT mutex...
Expected Counter = 400000
Actual Counter = 287543

Running WITH mutex...
Expected Counter = 400000
Actual Counter = 400000
*/

Brief Explanation

Race Condition: The statement `counter++` is not atomic. It involves reading the current value, incrementing it, and writing it back. When multiple threads execute this operation simultaneously, updates can be lost because threads may read the same value before either writes the result back.

Mutex Solution: A mutex ensures that only one thread enters the critical section at a time. `pthread_mutex_lock()` acquires the lock before updating the shared counter, and `pthread_mutex_unlock()` releases it afterward. This prevents concurrent modification of the shared variable and guarantees the correct final value.